
DAWN-SRW: A Computational Model of Learned State Transitions

Seungho Choi¹

Abstract

At the most general level, a state-transition system can be described by a state space, a transition rule or family of transition rules, and a transition protocol specifying how those transitions are applied to form a trajectory. Observable computational systems add initialization and readout maps: an external input or initial condition initializes a state, a transition protocol applies transition rules to evolve that state, and an observable output is read from the resulting state. For a language model, an input sequence initializes a residual representation state, a fixed transition protocol—the layer schedule—applies a finite sequence of learned layer-wise transitions, and a readout produces the token distribution. We propose the DAWN-CEU state-transition formulation, which factorizes individual transition rules into Control, Execution, and Update maps. We specialize this formulation to Operator-Coordinate CEU, in which Control maps a representation state into an operator space and parameterizes a sparse state-to-operator execution relation.

DAWN-SRW (DAWN Select–Read–Write) is a learned residual-state instance of this specialization. Select produces an operator-coordinate execution specification, RWCompose executes explicit rank-1 read/write transition operators, and a fixed residual update integrates the resulting state change. The current v4.1.6.7 implementation additionally introduces Global CEU Operator Pages: a page-routed candidate-retrieval mechanism that scores fixed-size operator blocks by live centroids in operator space and then applies the same DirectTau read/write execution inside the selected candidate pages. A completed 40M v4.1.6.6 operator-key run trains on 5B C4 tokens and reaches validation loss 3.6553 and accuracy 34.73%, compared with 3.5986 and 35.44% for a parameter-matched dense Transformer baseline.

Under execution pruning at $\epsilon = 10^{-1}$, validation loss remains 3.6552 with an estimated compute fraction of 0.0424 under the pruning accounting. A legacy v4.1.5.5 route-embedding implementation provides an additional empirical anchor for hard-pruned execution and context-dependent RW-neuron traces, and an earlier 400M SRW-family pilot provides preliminary scaling and transfer evidence. Overall, DAWN-SRW trains as a language model with observable control outputs, traceable execution, and a concrete path toward candidate-restricted sparse operator execution.

1. Introduction

Dense transformations are a central substrate of modern neural architectures. They are expressive, stable to optimize, and efficient on contemporary accelerators. However, this computational form is also opaque: a dense layer maps an input representation to an output representation through parameterized operations, without producing an observable account of which identifiable operations were selected, executed, and composed for a given input.¹

The distinction from existing conditional-computation architectures is therefore not sparsity alone. A dense Transformer learns a global dense transition over the representation state. A Mixture-of-Experts model routes states to sparse opaque expert modules. DAWN-SRW routes states to explicit read-write transition operators, forming a functional transition-operator space and state-dependent execution domains.

At the most general level, a state-transition system is described by a state space $\mathcal{S}$, a transition rule or family $\mathcal{T}$, and a transition protocol Π specifying how rules are applied to form trajectories:

$$\mathcal{D} = (\mathcal{S}, \Pi, \mathcal{T}). \quad (1)$$

Here $\mathcal{T}$ denotes a transition family and $T \in \mathcal{T}$ an individual transition rule; Π governs trajectory formation, such as fixed schedules, recurrence, conditional application, external interaction, or termination.

¹Code is available at <https://github.com/madst0614/dawn-srw>.

¹Independent Researcher. Correspondence to: Seungho Choi <madst0614@gmail.com>.

Observable input-output systems add initialization and read-out maps:

$$\mathcal{M} = (\mathcal{E}, \mathcal{S}, \mathcal{O}; \Phi, \Pi, \mathcal{T}, H), \quad (2)$$

where $\mathcal{E}$ and $\mathcal{O}$ are input and output spaces, Φ initializes s_0 from e , Π applies transitions from $\mathcal{T}$ to form a trajectory, and H reads the resulting state:

$$s_0 = \Phi(e), \quad s_0 \xrightarrow{\Pi, \mathcal{T}} s_1 \xrightarrow{\Pi, \mathcal{T}} \dots \xrightarrow{\Pi, \mathcal{T}} s_*, \quad o = H(s_*) \quad (3)$$

The unconditioned form $s_{t+1} = T(s_t)$ is the special case of repeatedly applying a single transition rule.

For a language model, the initialization map is an encoding map that maps an input sequence into an initial residual representation state,

$$s_0 = \Phi_\theta(x), \quad (4)$$

where Φ_θ includes token embedding, positional information, and initial residual-state construction. The transition protocol is fixed by the architecture: a finite sequence of learned layer-wise transition maps is applied,

$$\Pi_{\text{fixed}} = (T_{\theta,1}, T_{\theta,2}, \dots, T_{\theta,L}). \quad (5)$$

This yields

$$s_L = T_{\theta,L} \circ \dots \circ T_{\theta,1}(s_0), \quad (6)$$

and the output map is a readout map from the final state to a token distribution,

$$p(y | x) = R_\theta(s_L). \quad (7)$$

Thus standard fixed-depth language models are a special case in which the transition protocol is a fixed layer schedule, while the internal computation is a learned trajectory over residual representation states.

DAWN-CEU factorizes the transition rule itself, not the transition protocol. The transition protocol Π determines how transition rules are applied to form a trajectory; CEU decomposes an individual transition into Control, Execution, and Update components. Given a transition rule $T \in \mathcal{T}$ applied under a transition protocol Π , DAWN-CEU factorizes T as

$$T(s) = U(s, E(s, C(s))). \quad (8)$$

At the general, not necessarily learned, level, we denote a CEU factorization by

$$\mathfrak{F} = (\mathcal{S}, \mathcal{U}_{\text{spec}}, \mathcal{Z}; C, E, U), \quad (9)$$

where $\mathcal{U}_{\text{spec}}$ is an execution-specification space and $\mathcal{Z}$ is an execution-result space. Equivalently, given a current system state $s_t \in \mathcal{S}$, a DAWN-CEU transition proceeds as

$$u_t = C(s_t), \quad (10)$$

$$z_t = E(s_t, u_t), \quad (11)$$

$$s_{t+1} = U(s_t, z_t). \quad (12)$$

Here C is the control rule, which produces an execution specification u_t from the current state; E is the execution rule, which realizes an execution result z_t from the current state and specification; and U is the update rule, which applies that result to obtain the next state. Control should not be confused with the transition protocol: the protocol governs when and how transitions are applied, while the control rule operates inside a transition and specifies how that transition is executed. A transition without explicit control can be represented by a degenerate control rule that produces a fixed or null specification. Dense layers are not excluded from the DAWN-CEU formulation: they can also be written as state-transition maps, but typically correspond to implicit or unstructured cases in which no observable execution specification is produced.

Operator-coordinate CEU. In this specialization, the execution specification space is an operator-coordinate space. The control map sends the current representation state into this space, where coordinates parameterize a sparse relation between the state and a family of transition operators. Execution realizes the selected operators, and Update integrates the resulting state change back into the representation state. Formally, for a representation state $x \in \mathcal{S}$,

$$C : \mathcal{S} \rightarrow \mathcal{Z}_{\text{op}}, \quad z_{\text{op}} = C(x), \quad (13)$$

where $\mathcal{Z}_{\text{op}}$ is an operator-coordinate space. The coordinate z_{op} does not directly produce the next state; it parameterizes a sparse execution relation over a transition-operator family.

For a fixed execution specification $u \in \mathcal{U}_{\text{spec}}$, the execution map induces an execution function

$$f_u : \mathcal{S} \rightarrow \mathcal{Z}, \quad f_u(s) = E(s, u). \quad (14)$$

We call the resulting ability *state-transition traceability*: the ability to follow a transition through exposed CEU objects and the execution function induced by its specification. The CEU formulation alone does not imply interpretability; detailed space and function traceability conditions are given in Appendix A.3.

Throughout, subscripts θ indicate learned parameterization; fixed architectural rules are written without θ . A learned computation is therefore not the general formulation itself, but a parameterized instance of it:

$$u_t = C_{\theta_C}(s_t), \quad (15)$$

$$z_t = E_{\theta_E}(s_t, u_t), \quad (16)$$

$$s_{t+1} = U_{\theta_U}(s_t, z_t), \quad (17)$$

with $\theta = (\theta_C, \theta_E, \theta_U)$. This induces a learned transition rule

$$T_\theta(s) = U_{\theta_U}(s, E_{\theta_E}(s, C_{\theta_C}(s))). \quad (18)$$

When the update rule is fixed by the architecture, the learned transition instead takes the form

$$T_\theta(s) = U(s, E_{\theta_E}(s, C_{\theta_C}(s))). \quad (19)$$

Rules fixed by the architecture can be viewed as having empty or singleton parameter spaces. Thus, the DAWN-CEU formulation separates the structural skeleton of a transition from the learnable parameter dimensions that instantiate it.

DAWN-SRW is not the general state-transition system itself; it is an explicit residual-state implementation of Operator-Coordinate CEU. Its transition protocol in the fixed-depth experiments is the model’s layer schedule, and its state is the residual representation $s_t = x_t$:

$$u_t = \text{Select}_{\theta_C}(x_t), \quad (20)$$

$$z_t = \text{RWCompose}_{\theta_E}(x_t, u_t). \quad (21)$$

In DAWN-SRW, the update rule is the fixed residual update,

$$x_{t+1} = U(x_t, z_t) = x_t + z_t, \quad (22)$$

so $z_t = \Delta x_t$ is the residual state change.

The DAWN-CEU formulation does not require a particular control mechanism, execution mechanism, update rule, or unit decomposition. DAWN-SRW adds a learned functional read-write transition-operator family stored in reusable RW-neuron pools. Each RW operator implements the minimal rank-1 read/write map

$$W_i(x) = O_i^{RW}(x) = \langle x, r_i \rangle w_i, \quad (23)$$

where r_i reads a scalar component of the current residual state and w_i writes a direction of change back into the residual stream. In the v4.1.6.6 formulation, the routing key of an operator is derived from its own read/write directions,

$$k_i = K(r_i, w_i), \quad (24)$$

tying the control-facing identity of the operator to its execution-facing read/write map. Select compares the current operator-coordinate query with the RW-derived operator key,

$$\rho_i(x) = \cos(C(x), k_i), \quad (25)$$

and produces a soft gate from the boundary relation

$$m_i(x) = \rho_i(x) - \tau(x), \quad \gamma_i(x) = \text{admission}_i(x) \text{ drive}_i(x). \quad (26)$$

Schedule and gate-shape details are implementation choices and are kept in the method and appendix. With $\mathcal{O}_{RW} = \{W_i\}$, the execution and full residual transition are

$$E(x, C(x)) = \sum_i \gamma_i(x) \langle x, r_i \rangle w_i, \quad (27)$$

$$U(x, \Delta x) = x + \Delta x, \quad (28)$$

$$T(x) = x + \sum_i \gamma_i(x) \langle x, r_i \rangle w_i. \quad (29)$$

Thus, DAWN-SRW learns a representation geometry, a functional read-write transition-operator family, and a sparse state-to-operator execution relation between representation states and explicit transition operators.

Because routing is soft, these relations should be read as overlapping soft domains rather than disjoint partitions. For an operator W_i , its ϵ -active execution domain is

$$\Omega_i^\epsilon = \{x \in \mathcal{S} \mid \gamma_i(x) > \epsilon\}. \quad (30)$$

The resulting routing-induced execution domains form an overlapping soft domain decomposition over the residual state space.

For a given Select output, the induced execution function takes the read/write form

$$f_{u_t}(s) = \sum_{i \in A(x_t)} \gamma_i(x_t) \langle s, r_i \rangle w_i. \quad (31)$$

Evaluated at $s = x_t$, this gives $z_t = f_{u_t}(x_t)$ and the residual transition

$$x_{t+1} = x_t + f_{u_t}(x_t) = x_t + \sum_{i \in A(x_t)} \gamma_i(x_t) \langle x_t, r_i \rangle w_i. \quad (32)$$

Thus the transition rule is traceable through the selected read/write component functions.

Each DAWN-SRW block consists of an Attention Layer followed by a Residual State Transition (RST) Layer. Attention contextualizes token states using Select/RWCompose-constructed Q, K, and V; the RST Layer then applies a gate-normalized read/write transition over the RST pool to move the residual state forward.

DAWN-SRW inherits DAWN’s signature–operation abstraction while instantiating it under the DAWN-CEU state-transition formulation: operator identities serve Select, operations become read/write maps for RWCompose, and residual addition applies the execution output. Appendix A compares DAWN, the formulation, and DAWN-SRW; Appendix A.4 shows how dense attention fits the formulation while leaving dense interface construction implicit and semantically entangled.

This draft is organized around the current operator-key and page-based formulation. The completed v4.1.6.6 experiment is used as the primary 40M validation of the RW-derived operator-key path. The v4.1.6.7 page implementation is described as the current scalable extension of the same operator-coordinate mechanism; its final loss and wall-clock trade-off are not yet claimed. The earlier v4.1.5.5 route-embedding run is retained as a legacy preliminary pilot because it provides the strongest completed package of hard-pruned evaluation and context-dependent RW-neuron trace analysis. We also retain a 400M earlier SRW-family pilot for scaling and downstream transfer evidence.

Our contributions are as follows:

- **DAWN-CEU state-transition formulation.** We propose the DAWN-CEU formulation over general state-transition systems: a transition protocol forms trajectories from transition rules, and CEU factorizes individual transition rules into Control, Execution, and Update maps.
- **Operator-coordinate DAWN-SRW architecture.** We instantiate the formulation over the residual representation space using Select as the control rule, RWCompose as the execution rule, and a fixed residual update as the update rule. DAWN-SRW learns a sparse state-to-operator execution relation over explicit read/write transition operators.
- **RW-derived operator-key Select.** The current DAWN-SRW formulation ties the Select-facing identity of each operator to its read/write operation by deriving operator keys from the same read and write directions that execute the transition. The completed v4.1.6.6 run validates that this final operator-key path trains at 40M scale.
- **Global CEU Operator Pages.** The v4.1.6.7 implementation adds a coarse page-retrieval layer over the global operator atlas. Pages are fixed-size candidate blocks scored by live centroids in operator space, while DirectTau and RWCompose still determine token-specific operator execution inside the selected blocks.
- **Preliminary execution and trace evidence.** We report 40M operator-key validation, legacy hard-pruned execution and RW-neuron trajectory analysis, and an earlier 400M SRW-family scaling and transfer pilot. We explicitly separate structural sparsity and theoretical pruning accounting from measured wall-clock speedup.

2. Related Work

Transformer computation and residual representations. Transformer language models update token representations through repeated attention and feedforward transformations (Vaswani et al., 2017). Mechanistic analyses often describe these models in terms of a residual stream that carries information between layers (Elhage et al., 2021). In standard architectures, however, the transformations applied to this stream are implemented by dense parameterized maps whose internal operation structure is not directly observable as selected and executed units. DAWN-SRW starts from the same residual-stream perspective on language modeling, but represents computation through the DAWN-CEU formulation over RW neurons. The goal is not only to produce

an output, but to make observable which RW neurons were selected and how they were gated, and to make traceable how their execution contributed to the residual-state change.

Conditional computation and sparse expert models. Mixture-of-Experts models route tokens to a small number of expert networks (Shazeer et al., 2017; Fedus et al., 2022). These methods introduce conditional computation at the level of large subnetworks, typically whole feedforward experts. Under the CEU view, MoE layers are space-traceable at the expert-routing level because the router exposes selected experts and gates; however, unless the experts expose their internal computation, the induced execution function remains opaque inside each dense expert. DAWN-SRW uses a finer-grained form of control: Select maps states into an operator-coordinate space, compares them with explicit RW operator keys, and RWCompose composes the resulting read/write transitions into an output. The selected unit is not a full expert block, but a minimal read/write transition operator that reads a scalar component of the current state and writes a learned direction of change.

Low-rank and factorized transformations. Low-rank methods such as LoRA introduce trainable low-rank updates to dense weights, often for parameter-efficient adaptation (Hu et al., 2022). DAWN-SRW also uses factorized structure, but in a different role. RW neurons are not auxiliary updates to a frozen dense model; they are the primary computational substrate. Each RW neuron executes a rank-1 transition $O_i(x) = \langle x, r_i \rangle w_i$, and different RW neurons are selected for different states.

State-space models and learned dynamics. State-space models formulate sequence modeling as the evolution of a hidden state over time, and selective SSMs make the transition dynamics input-dependent (Gu et al., 2022; Gu & Dao, 2024). DAWN-SRW shares the state-transition perspective, but applies it along a different axis. SSMs primarily model temporal memory dynamics along the sequence dimension: what information should be propagated, forgotten, or decayed across tokens. DAWN-SRW models residual-state computation across layers: each layer treats the residual representation as the current state and moves it by selecting RW neurons that transform it into the next computation state. Thus, DAWN-SRW is not a replacement for sequence recurrence, but a model of how learned transformations themselves can be decomposed through explicit DAWN Select–Read–Write transitions.

Post-hoc interpretability and architectural observability. Sparse autoencoders and related interpretability tools discover features in the activations of already-trained dense models (Cunningham et al., 2023). DAWN-SRW instead builds identifiable components into the architecture. A RW

neuron has an explicit read/write transition operation and, in the final operator-coordinate formulation, a routing key derived from that operation, allowing the model’s internal computation to be analyzed as control over identifiable transition operators followed by executed contributions. This does not remove the need for analysis, but it changes the object of analysis from post-hoc discovered features to architecturally observable selection traces and traceable execution contributions.

From DAWN to DAWN-SRW. DAWN (Choi, 2025) is the closest predecessor to DAWN-SRW: it introduced the signature–operation abstraction and replaced Transformer projection matrices with dynamic compositions of low-rank Feature–Restore neurons from shared pools. DAWN-SRW retains the abstraction but recasts it under the DAWN-CEU state-transition formulation: the control-facing identity is tied to an explicit read/write transition operator, execution becomes a composition of rank-1 read/write maps, and the transition is implemented through Select, RWCompose, and a fixed residual update. Thus, DAWN-SRW is not a direct reimplement of the original DAWN architecture, but a CEU-factorized residual-state implementation.

3. Method

The method instantiates Operator-Coordinate CEU as a learned residual-state model. The current method has a two-stage operator-selection path. First, the v4.1.6.6 operator-key path maps a residual state into an operator-coordinate query, compares that query with RW-derived operator keys, and executes selected read/write transition operators through DirectTau admission and RWCompose. Second, the v4.1.6.7 page path adds a coarse candidate-retrieval stage over the same global operator atlas: it ranks fixed-size operator pages by live centroid similarity before applying the original operator-level DirectTau and read/write execution inside the selected candidate blocks. Thus v4.1.6.7 should be read as the page-routed extension of the v4.1.6.6 operator-key formulation, not as a separate expert architecture.

3.1. DAWN-SRW as a Learned Residual-State Instance

In DAWN-SRW, the abstract system state s_t is instantiated as the residual representation state x_t . At each layer and token position, the residual-stream vector is treated as the current residual state. DAWN-SRW instantiates the CEU control rule as Select, the execution rule as RWCompose, and the update rule as a fixed residual update:

$$u_t = \text{Select}_{\theta_C}(x_t), \quad (33)$$

$$z_t = \text{RWCompose}_{\theta_E}(x_t, u_t), \quad (34)$$

$$x_{t+1} = U(x_t, z_t) = x_t + z_t. \quad (35)$$

Here u_t identifies participating RW operators and gates, and $z_t = \Delta x_t$ is the composed residual state change. The equations in this section describe the current v4.1.6.7 method structure. The completed 40M result in Section 4.1 evaluates the full-pool v4.1.6.6 operator-key path; Section 4.4 describes the v4.1.6.7 page-routed implementation status.

For a single RW-neuron pool p , Select produces a pool execution specification

$$u_p(x_t) = \text{Select}_{\theta}^p(x_t) = \{(i, \gamma_i^p(x_t))\}_{i=1}^{N_p}, \quad (36)$$

where $\gamma_i^p(x_t)$ is the gate assigned to RW operator i . Each selected RW operator has a read/write operation

$$O_i^{RW,p}(x_t) = \langle x_t, r_i^p \rangle w_i^p. \quad (37)$$

RWCompose then gates, normalizes, and sums these RW contributions:

$$y_p(x_t) = \alpha_p \frac{\sum_{i=1}^{N_p} \gamma_i^p(x_t) \langle x_t, r_i^p \rangle w_i^p}{\max\left(\sum_{i=1}^{N_p} \gamma_i^p(x_t), 1\right)}. \quad (38)$$

The residual update is fixed by the architecture; the learnable components are the control and execution parameterizations.

3.2. Operator Space and RW-Derived Operator Keys

DAWN-SRW separates the representation space in which token states live from the operator space in which transition operators are addressed. The model does not route directly over representation features. It first maps the current residual state into an operator-coordinate query and then selects read/write transition operators near that coordinate.

Each RW operator is an identifiable transition unit with an execution-facing read/write map and a control-facing operator key. For operator i , let

$$r_i, w_i \in \mathbb{R}^{d_{\text{model}}}, \quad k_i \in \mathbb{R}^{d_{\text{route}}}. \quad (39)$$

The read/write operation is

$$O_i^{RW}(x) = \langle x, r_i \rangle w_i. \quad (40)$$

The read vector extracts a scalar component from the current residual state, and the write vector maps that scalar into a direction of residual-state change.

In the operator-key formulation, the Select-facing key is derived from the RW operation itself:

$$k_i = K_{\theta}(r_i, w_i). \quad (41)$$

The implementation uses normalized read and write directions, projects them into the route dimension, multiplies the two compressed signatures elementwise, and normalizes the

result. This ties the control-facing identity of an operator to the same read/write directions that execute the transition.

The current operator query is also RW-matched. A read-side query is produced from the residual state, and a write-side query is produced by projecting the same state into the operator route dimension. The resulting query has the form

$$h(x) = \text{normalize}(\text{normalize}(q_r(x)) \odot \text{normalize}(q_w(x))), \quad (42)$$

where $\odot$ denotes elementwise multiplication. Select compares $h(x)$ with RW-derived operator keys by cosine similarity:

$$\rho_i(x) = \langle \text{normalize}(h(x)), \text{normalize}(k_i) \rangle. \quad (43)$$

3.3. DirectTau Admission and Boundary-Relative Drive

The operator-key path uses a cosine-space DirectTau boundary. A learned raw threshold is mapped into the cosine interval,

$$\tau(x) = -1 + 2\sigma(\tau_{\text{raw}}(x)), \quad (44)$$

and the selection margin is

$$m_i(x) = \rho_i(x) - \tau(x). \quad (45)$$

DirectTau separates whether an operator is admitted near the boundary from how strongly it is driven once admitted:

$$\gamma_i(x) = \text{admission}_i(x) \text{drive}_i(x). \quad (46)$$

The admission term is a one-sided boundary function of the negative margin, and the drive term is a boundary-relative positive contribution scale. The main point is structural: Select produces an observable state-to-operator execution relation, while RWCompose executes the selected read/write maps. Schedule details, boundary temperature, boundary power, and execution-pruning thresholds are implementation choices.

3.4. RWCompose: Executing the Selected Transition

RWCompose receives the current state and the execution specification produced by Select. For a candidate operator set $\mathcal{C}(x)$, which may be the full pool or the page-retrieved candidate set, the composed output is

$$y_p(x) = \alpha_p \frac{\sum_{i \in \mathcal{C}(x)} \gamma_i^p(x) \langle x, r_i^p \rangle w_i^p}{\max \left(\sum_{i \in \mathcal{C}(x)} \text{admission}_i^p(x), 1 \right)^\eta}, \quad (47)$$

where η is the admission-denominator power used by the implementation. The v4.1.6.6 full-pool path takes $\mathcal{C}(x)$ to be the full pool. The v4.1.6.7 page-routed path first constructs $\mathcal{C}(x)$ from selected operator pages and then applies the same DirectTau/RWCompose equations inside that candidate set.

For an operator W_i , the soft gate induces an ϵ -active execution domain

$$\Omega_i^\epsilon = \{x \in \mathcal{S} \mid \gamma_i(x) > \epsilon\}. \quad (48)$$

These routing-induced execution domains may overlap across operators. They should therefore be understood as an overlapping soft domain decomposition rather than a disjoint partition.

3.5. Global CEU Operator Pages

The global operator atlas gives DAWN-SRW an explicit set of reusable transition operators, but a full-pool implementation still compares each query with every operator. Global CEU Operator Pages add a coarse candidate-retrieval layer over the same operator space.

Let the operator pool be partitioned into fixed-size pages P_j . Each page contains a fixed-size block of RW transition operators from the global operator pool. In the current implementation, pages are fixed candidate blocks rather than learned clusters or periodically repacked semantic regions. For page P_j , define a live page centroid from the RW-derived operator keys in that block:

$$c_j = \text{normalize} \left(\frac{1}{|P_j|} \sum_{i \in P_j} k_i \right). \quad (49)$$

Given an operator query $h(x)$, page scores are computed by cosine similarity,

$$s_j(h) = \langle \text{normalize}(h), c_j \rangle. \quad (50)$$

The model selects the top- K nearest pages and forms a temporary candidate operator set

$$\mathcal{C}_{\text{page}}(h) = \bigcup_{j \in \text{TopK}(s(h))} P_j. \quad (51)$$

DirectTau admission and RWCompose execution are then applied only inside $\mathcal{C}_{\text{page}}(h)$.

This distinction is important. Page selection is a coarse, Voronoi-like neighborhood selection over live page centroids in operator space. It should not be read as a claim that operators inside each fixed page already form a learned semantic cluster. A page is not an expert layer and it does not itself perform the computation. The actual computation is still performed by token-specific read/write transition operators selected and weighted by DirectTau inside the selected candidate pages. In the current implementation, page selection may be shared by a small microgroup for efficient dense candidate execution, but each token retains its own operator query, threshold, admission, drive, and read/write contribution weights inside the selected candidate set.

3.6. DAWN-SRW Block: Attention Followed by RST

Each DAWN-SRW block consists of an Attention Layer followed by a Residual State Transition (RST) Layer. Both apply Select and RWCompose, but for different roles. The Attention Layer uses Select and RWCompose to construct Q, K, and V for relational token interaction. The RST Layer uses Select and RWCompose to produce the post-attention residual-state transition.

Attention Layer. Given a residual state $x_{\ell,t}$, the Attention Layer first applies normalization and constructs Q, K, and V through attention pools. Query and key use the shared attention-QK pool but make separate Select control passes; value uses the attention-V pool:

$$Q_{\ell,t} = \text{RWCompose}_{\theta}^Q(x_{\ell,t}, \text{Select}_{\theta}^Q(x_{\ell,t})), \quad (52)$$

$$K_{\ell,t} = \text{RWCompose}_{\theta}^K(x_{\ell,t}, \text{Select}_{\theta}^K(x_{\ell,t})), \quad (53)$$

$$V_{\ell,t} = \text{RWCompose}_{\theta}^V(x_{\ell,t}, \text{Select}_{\theta}^V(x_{\ell,t})). \quad (54)$$

The resulting Q, K, and V are passed through standard causal self-attention to produce the attention output $a_{\ell,t}$, and the intermediate residual state is

$$\tilde{x}_{\ell,t} = x_{\ell,t} + a_{\ell,t}. \quad (55)$$

RST Layer. The RST Layer receives the contextualized state $\tilde{x}_{\ell,t}$ and applies the explicit DAWN-SRW transition directly:

$$u_{\ell,t}^{RST} = \text{Select}_{\theta}^{RST}(\tilde{x}_{\ell,t}), \quad (56)$$

$$\Delta x_{\ell,t}^{RST} = \text{RWCompose}_{\theta}^{RST}(\tilde{x}_{\ell,t}, u_{\ell,t}^{RST}), \quad (57)$$

$$x_{\ell+1,t} = \tilde{x}_{\ell,t} + \Delta x_{\ell,t}^{RST}. \quad (58)$$

Select determines which RST-pool operators participate, and RWCompose executes their gate-weighted read/write transition. The Attention Layer therefore contextualizes token states, while the RST Layer moves the contextualized state to the next residual state.

3.7. Implementation Roles of v4.1.5.5, v4.1.6.6, and v4.1.6.7

The paper separates three implementation roles. The v4.1.6.6 implementation is the completed full-pool operator-key path and is used as the primary 40M validation of the final RW-derived-key formulation. The v4.1.6.7 implementation keeps the same operator-key Select and Direct-Tau/RWCompose execution but adds Global CEU Operator Pages for page-routed candidate execution. Its current role in this draft is architectural and implementation-level: final page capacity, validation loss, and wall-clock trade-offs are not claimed until the page run completes. The older v4.1.5.5 implementation is retained as a legacy preliminary pilot because it provides completed hard-pruned evaluation and

Model	Eval	Params	Loss	Acc.	Exec. note
Dense TF	full	40.344M	3.5986	35.44	dense
DAWN v4.1.6.6	full	40.220M	3.6553	34.73	full pool
DAWN v4.1.6.6	prune $\epsilon = 10^{-1}$	–	3.6552	34.73	0.0424 est.

Table 1. Primary 40M operator-key validation. The execution note for the pruned row is the estimated compute fraction reported by the pruning accounting, not a measured wall-clock speedup and not directly comparable to the dense baseline row.

context-dependent trace analysis under a route-embedding Select mechanism.

4. Experiments

We report experiments in four roles. First, the completed 40M v4.1.6.6 run is the primary validation that the RW-derived operator-key formulation trains as a language model. Second, the legacy 40M v4.1.5.5 route-embedding implementation is retained as a preliminary empirical anchor for hard-pruned execution and context-dependent RW-neuron traces. Third, the v4.1.6.7 page implementation is described as the current scalable extension, with final empirical page results left for the completed version. Fourth, a 400M earlier SRW-family DAWN pilot is summarized for scaling and transfer evidence.

4.1. 40M Operator-Key Validation

We train DAWN-SRW v4.1.6.6 and a dense Transformer baseline on the same 5B-token C4 budget for 76,293 steps. The DAWN-SRW run has 40.220M parameters and uses the full-pool RW-derived operator-key path: live-gradient RW keys, RW-matched operator queries, DirectTau admission, boundary-relative drive, and read/write composition. The dense Transformer baseline has 40.344M parameters and uses a larger residual width with a standard feed-forward layer.

The v4.1.6.6 model trains to completion and reaches validation loss 3.6553 with validation accuracy 34.73%. The dense Transformer baseline reaches validation loss 3.5986 with validation accuracy 35.44%. Thus the completed operator-key run is not presented as outperforming the dense baseline; its role is to validate that the final RW-derived operator-key formulation remains trainable at 40M scale and supports sparse execution pruning without loss degradation under the reported pruning accounting.

At end-of-epoch validation, execution pruning at $\epsilon = 10^{-2}$ gives loss 3.6552 with estimated compute fraction 0.0426, and execution pruning at $\epsilon = 10^{-1}$ gives loss 3.6552 with estimated compute fraction 0.0424. Both rows leave validation loss effectively unchanged relative to the full v4.1.6.6 evaluation. This supports the limited claim that

Execution	Loss	Acc.	RST kept	RW-only
Dense TF	3.60568	35.397	–	–
DAWN v4.1.5.5 full	3.62589	35.153	20,560	1.000×
DAWN v4.1.5.5 hard .99	3.62590	35.152	3,031	0.131×

Table 2. Legacy 40M v4.1.5.5 hard-pruned preliminary pilot. RW-only FLOPs are estimated selected-read/write execution costs, not measured wall-clock speedups.

the operator-key DirectTau/RWCompose path concentrates executed read/write computation into a small retained set under the pruning convention.

4.2. Legacy v4.1.5.5 Preliminary Pilot

We retain the v4.1.5.5 run as a legacy preliminary pilot because it provides a completed package of hard-pruned evaluation and RW-neuron trace analysis. This implementation predates the final RW-derived-key Select mechanism and uses legacy independent route embeddings. It should therefore not be read as an evaluation of the current v4.1.6.6/v4.1.6.7 operator-key path. It is used to show that the Select/RWCompose residual-state architecture trains, supports strict hard-pruned forward passes, and exposes traceable context-dependent RW-neuron trajectories.

Under the paper evaluation protocol, v4.1.5.5 reaches validation loss 3.62589 and accuracy 35.153%, compared with 3.60568 and 35.397% for its parameter-matched dense baseline. At the strict threshold $\rho = 0.99$, hard-pruned evaluation retains only about 13% of estimated RW execution while leaving validation loss unchanged within 10^{-5} . The detailed hard-pruning protocol and FLOPs accounting are given in Appendix E.

4.3. Context-Dependent RST Control Traces

The trace analysis is performed on the v4.1.5.5 legacy pilot. Our main analysis asks whether the RST Layer applies a fixed token-specific RW-neuron set or a context-dependent trace. For contextually underdetermined token states, we compare two short probe contexts with different plausible resolutions, e.g., financial *bank* versus river *bank*. At each layer, we compare Jaccard overlap for three RST sets: broad active neurons, top- k projected contributors, and effective-mass contributors. Figure 1 averages this overlap over *bank*, *charge*, *light*, and *apple*; formal definitions are given in Appendix G.

The mean trajectory shows a consistent pattern. In the earliest layers, the two contexts share many RST RW neurons, including high-contribution RW neurons. After the first few layers, the top- k and effective-mass contributor overlaps drop sharply and remain low. In contrast, the broad active set remains more overlapping. This pattern indi-

Model	SST-2	RTE	WiC	BoolQ
TF rand.	75.57	53.07	56.27	64.86
DAWN rand.	76.49	51.26	55.96	65.63
TF C4	93.69	68.95	69.75	78.13
DAWN C4	94.04	71.48	67.24	77.52

Table 3. 400M transfer pilot. Accuracies are percentages.

cates that DAWN-SRW does not simply apply a fixed token-specific RW-neuron set. Instead, the RST Layer maintains a broad shared candidate space while composing different high-contribution RW neurons as the residual state becomes contextualized.

Bank and execution traces. Appendices G.1 and G.2 provide the single-token *bank* case and a final-layer execution trace. They show the same pattern at finer resolution and distinguish routing evidence from executed contribution, which also depends on the read scalar, gate, write direction, and alignment with the composed RST output.

4.4. v4.1.6.7 Page-Routed Implementation Status

The v4.1.6.7 implementation adds Global CEU Operator Pages to the v4.1.6.6 operator-key path. The implementation supports page-enabled QK, V, and RST pools; page selection retrieves top- K fixed-size candidate pages according to live centroid similarity in operator space, and DirectTau/RWCompose is then applied inside the resulting candidate set. This section is intentionally not a final empirical comparison. At the time of this draft, v4.1.6.7 is included as the current method and implementation path, while final page-size, page-capacity, validation-loss, and wall-clock trade-off results remain pending.

4.5. 400M Scaling and Transfer Pilot

We also summarize a 400M-scale pilot with spatial-r1-v3.9.4, an earlier SRW-family DAWN model. This result is used only as scaling and transfer evidence, not as evidence for the v4.1.6.6 operator-key mechanism, the v4.1.6.7 page mechanism, or implemented sparse inference efficiency. Table 3 gives a compact downstream summary; full setup, checkpoint metrics, utilization diagnostics, and compute accounting are given in Appendix I.

These results support the limited claim that RW-neuron composition can scale beyond the 40M setting and learn transferable representations, while leaving final sparse-execution and page-retrieval claims to the later v4.1.6.6/v4.1.6.7 path.

5. Discussion and Limitations

The central architectural idea is that DAWN-SRW computes by locating a residual state in an operator space

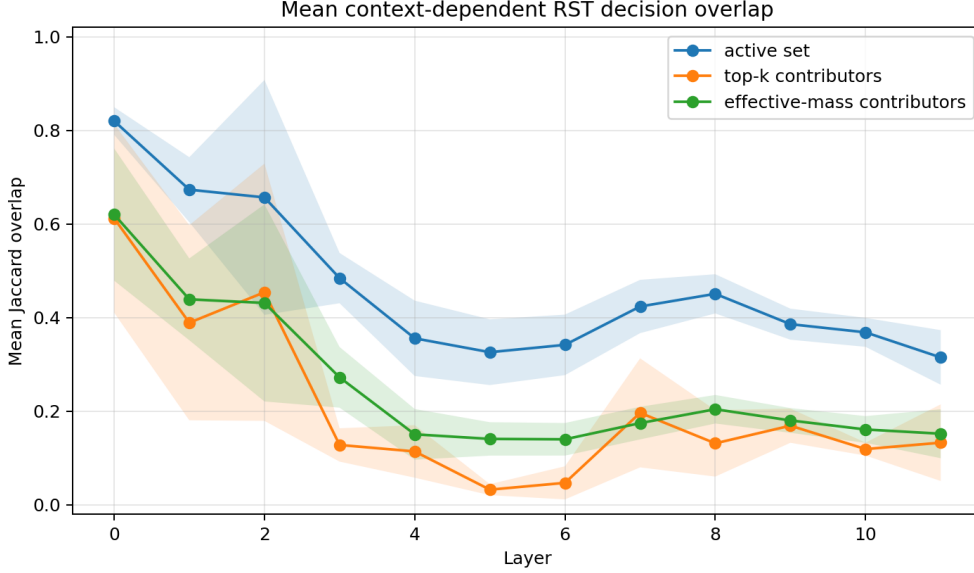


Figure 1. Mean context-dependent RST control/transition overlap for contextually underdetermined token states in the v4.1.5.5 legacy pilot. Broad active sets remain partially shared across contexts, while top- k projected contributors and effective-mass contributors diverge sharply after the early layers. This indicates that the RST Layer maintains a broad shared candidate space while forming context-dependent high-contribution RW-neuron compositions.

and executing selected read/write state-transition operators. The v4.1.6.6 result validates the completed RW-derived operator-key path at 40M scale. The v4.1.6.7 page mechanism then extends the same operator-coordinate structure into a candidate-retrieval architecture: fixed-size pages are ranked by live centroid similarity first, and the original token-specific DirectTau/RWCompose execution remains responsible for the actual computation.

Language modeling as residual-state transition. Under the DAWN-CEU state-transition formulation, language modeling can be interpreted as learning a sequence of transitions over residual representation states. Each block updates the current state so that, after repeated contextualization and residual-state transitions, the final representation better supports the next-token distribution under the output readout. DAWN-SRW makes this process explicit: Select exposes u_t as a state-to-operator execution relation, RWCompose produces $z_t = \Delta x_t$ through rank-1 read/write contributions, and the RST Layer applies this transition after attention contextualization.

Scalable operator-space execution. Global CEU Operator Pages provide an implementation path from traceable sparse operators to practical candidate-restricted execution. A full-pool DAWN-SRW layer compares each operator query against the entire global RW-operator atlas. The page mechanism first ranks fixed-size pages by live centroid similarity in operator space and then applies DirectTau

admission and read/write composition only over selected candidate pages. This preserves the global operator-atlas interpretation while avoiding the requirement that every token scan every operator at execution time. The current draft treats this as an architectural and implementation extension; final wall-clock efficiency, page capacity, page geometry, and validation-loss trade-offs still need to be measured.

Traceability and empirical roles. The v4.1.5.5 trace analysis shows a contrast between broad candidate RW neurons and effective transition contributors: active RST sets remain moderately overlapping across contexts, but top projected and effective-mass contributor sets diverge after the early layers. Because that run uses legacy route embeddings, it is not evidence for the final RW-derived-key mechanism. Its role is narrower: it demonstrates the type of trace analysis enabled by Select/RWCompose and motivates the same analysis on the v4.1.6.6/v4.1.6.7 path after the current runs complete.

Scope of the general formulation. The general state-transition form is intentionally broad: it can describe discrete machines, continuous dynamical systems, biological regulatory systems, recurrent computation, and fixed-depth neural architectures at the level of state, transition, and protocol. DAWN-CEU does not claim that all such systems are interpretable; it factorizes a transition rule into roles whose structure may be fixed, learned, exposed, or implicit. DAWN-SRW realizes one explicit and traceable

neural language-model instance.

Several limitations remain. The v4.1.6.6 operator-key run is a completed 40M validation but does not outperform the dense Transformer baseline. The v4.1.6.7 page mechanism is implemented but not yet reported as a final empirical comparison. The context-dependence analysis is observational and currently comes from the v4.1.5.5 legacy pilot; same-sense controls, random-token baselines, sensitivity checks, and interventions are needed to establish causal necessity. The 400M results use spatial-r1-v3.9.4, an earlier SRW-family implementation, and should be read only as scaling and transfer evidence. Finally, structural sparsity and pruning-accounting estimates do not imply measured wall-clock speedup. Realizing computational benefits will require optimized retrieval, dispatch, sparse read/write kernels, and hardware-aware implementations.

Externally conditioned state-transition computation.

External conditions may enter the hierarchy at three levels: initialization, transition protocol, and transition-rule factorization. An external input or initial condition may initialize the state through Φ ; external signals may condition the transition protocol Π , changing which transition rules are applied or how the trajectory is formed; and external signals may condition the CEU maps C , E , and U of an individual transition. The present DAWN-SRW instantiation is internally conditioned and fixed-protocol; externally conditioned DAWN-CEU computation is a natural extension.

6. Conclusion

DAWN-SRW instantiates the DAWN-CEU state-transition formulation through Operator-Coordinate CEU, Select, RW-Compose, and a fixed residual update. The current formulation maps residual states into an operator-coordinate space, selects RW-derived read/write transition operators, and executes traceable rank-1 residual-state updates. The completed v4.1.6.6 40M run validates that the final RW-derived operator-key path trains to completion and preserves validation loss under aggressive execution pruning. The v4.1.6.7 implementation extends this path with Global CEU Operator Pages, which retrieve fixed-size candidate blocks from the global operator atlas before applying the original token-specific DirectTau and read/write execution. Legacy v4.1.5.5 and 400M earlier SRW-family pilots provide preliminary evidence for hard-pruned execution, trace analysis, scaling, and transfer. Together, these results support DAWN-SRW as a trainable language-model architecture with observable control outputs, traceable execution dynamics, and a concrete route toward scalable sparse operator-space computation.

Impact Statement

This paper presents work whose goal is to advance neural network architecture design, learned computation, and interpretability. By structuring model computation around identifiable RW-neuron operations, explicit control, and executed transition contributions, this work may contribute to better tools for analyzing internal model behavior and for studying efficient conditional computation. There are many potential societal consequences of neural language models; none are specific enough to this architectural study to require separate discussion here.

Acknowledgments

The author thanks Anthropic Claude, Claude Code, and OpenAI ChatGPT for assistance throughout this research. This work was developed with extensive use of AI systems for literature exploration, conceptual exploration, cross-domain framing, abstraction and terminology refinement, implementation support, debugging, and manuscript preparation. In particular, these systems were used as interactive assistants for refining the DAWN-CEU state-transition formulation and the exposition of DAWN-SRW. The research direction, architectural decisions, experimental design, experiments, analyses, interpretations, and final claims remain the responsibility of the author.

This research was supported by the Google TPU Research Cloud (TRC) program, which provided computational resources for training and analysis.

References

- Choi, S. DAWN: Dynamic subspace composition for language modeling. *Zenodo*, 2025. doi: 10.5281/zenodo.19599530. URL <https://doi.org/10.5281/zenodo.19599530>.
- Cunningham, H., Ewart, A., Riggs, L., Huben, R., and Sharkey, L. Sparse autoencoders find highly interpretable features in language models. In *International Conference on Learning Representations*, 2023.
- Elhage, N., Nanda, N., Olsson, C., Henighan, T., Joseph, N., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., DasSarma, N., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S., and Olah, C. A mathematical framework for transformer circuits. Transformer Circuits Thread, 2021. URL <https://transformer-circuits.pub/2021/framework/index.html>.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 2022.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. In *International Conference on Learning Representations*, 2024.
- Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

A. Terminology

This appendix collects the main terms used in the paper and gives a compact formalization of the state-transition-system view, transition protocols, CEU-factorized transitions, and traceability. We keep the distinction between general state-transition systems, observable input-output systems, transition protocols, learned DAWN-CEU instances, and the DAWN-SRW instantiation.

A.1. State-Transition System Hierarchy

At the most general level, a state-transition system may be written schematically as

$$\mathcal{D} = (\mathcal{S}, \Pi, \mathcal{T}), \quad (59)$$

where $\mathcal{S}$ is a state space, $\mathcal{T}$ is a transition rule or family of transition rules, and Π is a transition protocol. The protocol governs how transition rules are applied to configurations of the system to form a trajectory. It may encode a fixed schedule, recurrent iteration, conditional transition application, interaction with external signals, or termination of a transition sequence. Throughout this appendix, $\mathcal{T}$ denotes a transition family or a transition-rule family, while $T \in \mathcal{T}$ denotes an individual transition rule.

Observable input-output systems add initialization and readout maps:

$$\mathcal{M} = (\mathcal{E}, \mathcal{S}, \mathcal{O}; \Phi, \Pi, \mathcal{T}, H). \quad (60)$$

Here Φ initializes the system state from an external input or initial condition $e \in \mathcal{E}$, Π applies transition rules from $\mathcal{T}$ to form a trajectory, and H reads an observable output from the resulting state:

$$s_0 = \Phi(e), \quad s_0 \xrightarrow{\Pi, \mathcal{T}} s_1 \xrightarrow{\Pi, \mathcal{T}} \dots \xrightarrow{\Pi, \mathcal{T}} s_*, \quad o = H(s_*). \quad (61)$$

The familiar unconditioned state-space dynamics form is the special case in which a single transition rule is repeatedly applied under a simple transition protocol:

$$s_{t+1} = T(s_t), \quad T : \mathcal{S} \rightarrow \mathcal{S}. \quad (62)$$

For a language model, the initialization map is an encoding map into an initial residual representation state, the transition protocol is the fixed layer schedule, and the output map is a readout from the final state to a token distribution:

$$s_0 = \Phi_\theta(x), \quad (63)$$

$$\Pi_{\text{fixed}} = (T_{\theta,1}, T_{\theta,2}, \dots, T_{\theta,L}), \quad (64)$$

$$s_L = T_{\theta,L} \circ \dots \circ T_{\theta,1}(s_0), \quad (65)$$

$$p(y | x) = R_\theta(s_L). \quad (66)$$

Within this hierarchy, DAWN-CEU factorizes the transition rule itself, not the transition protocol. Given an individual transition rule $T \in \mathcal{T}$, the CEU factorization is

$$T(s) = U(s, E(s, C(s))), \quad (67)$$

with control, execution, and update maps producing

$$u = C(s), \quad z = E(s, u), \quad s' = U(s, z). \quad (68)$$

A learned DAWN-CEU instance parameterizes one or more of these rules:

$$T_\theta(s) = U_{\theta_U}(s, E_{\theta_E}(s, C_{\theta_C}(s))), \quad (69)$$

or, when the update is fixed by the architecture,

$$T_\theta(s) = U(s, E_{\theta_E}(s, C_{\theta_C}(s))). \quad (70)$$

DAWN-SRW is the residual-state instance in which $s_t = x_t$, Select supplies u_t , RWCompose supplies z_t , and $U(x, z) = x + z$:

$$u_t = \text{Select}_{\theta_C}(x_t), \quad (71)$$

$$z_t = \text{RWCompose}_{\theta_E}(x_t, u_t), \quad (72)$$

$$x_{t+1} = x_t + z_t. \quad (73)$$

In the operator-coordinate specialization, Select first maps the residual state into an operator-coordinate space and then parameterizes a sparse state-to-operator execution relation:

$$C : \mathcal{S} \rightarrow \mathcal{Z}_{\text{op}}, \quad z_{\text{op}} = C(x). \quad (74)$$

For DAWN-SRW, the transition-operator family is $\mathcal{O}_{RW} = \{W_i\}$, where

$$W_i(x) = \langle x, r_i \rangle w_i. \quad (75)$$

In the final v4.1.6.6 formulation, the Select key is derived from the RW operator itself:

$$k_i = K(r_i, w_i), \quad \rho_i(x) = \cos(C(x), k_i). \quad (76)$$

The gate $\gamma_i(x)$ is produced from the boundary relation between $\rho_i(x)$ and $\tau(x)$, and RWCompose executes

$$E(x, C(x)) = \sum_i \gamma_i(x) \langle x, r_i \rangle w_i. \quad (77)$$

For a fixed Select output u_t , RWCompose induces an execution function

$$f_{u_t}(s) = \sum_{i \in A(x_t)} \gamma_i(x_t) \langle s, r_i \rangle w_i, \quad (78)$$

so that $z_t = f_{u_t}(x_t)$ and $x_{t+1} = x_t + f_{u_t}(x_t)$.

A.2. Relationship between DAWN, DAWN-CEU, and DAWN-SRW

Aspect	DAWN	DAWN-CEU state-transition formulation	DAWN-SRW
Role	Original signature–operation architecture	Transition-factorization formulation over general state-transition systems	Explicit operator-coordinate residual-state instantiation
Control	Signature matching selects neurons	C produces an execution specification u_t	Select maps the state to an operator-coordinate query and outputs selected RW operators and gates
Execution	Selected operations form an input-dependent transformation	E produces an execution result z_t	RWCompose composes traceable RW-operation contributions into $z_t = \Delta x_t$
Update	Residual block update applies the transformation	U maps (s_t, z_t) to s_{t+1}	Fixed residual update $x_{t+1} = x_t + z_t$
Units	Identifiable low-rank neurons	Unit decomposition is not required by the formulation itself	RW neurons instantiate explicit read/write transition operators
Operation form	Rank- r Feature–Restore pathway	Execution may take many computational forms	Rank-1 read/write operation, $O_i^{RW}(x) = \langle x, r_i \rangle w_i$
Result	Input-dependent low-rank transformation within a residual block	State transition $s_t \rightarrow s_{t+1}$ through u_t, z_t, U	Explicit RW-composed residual state change $\Delta x_t = z_t$

Table 4. Relationship between DAWN, the DAWN-CEU state-transition formulation, and DAWN-SRW. DAWN-SRW retains DAWN’s signature–operation abstraction, but instantiates it as Operator-Coordinate CEU through Select, RWCompose, and a fixed residual update with traceable rank-1 read/write transition operators.

System-level terms.

Term	Meaning
General state-transition system	A system described by a state space $\mathcal{S}$, a transition rule or family of transition rules $\mathcal{T}$, and a transition protocol Π that specifies how transitions are applied to form a trajectory.
Observable state-transition system	A state-transition system with initialization and readout maps, written as $\mathcal{M} = (\mathcal{E}, \mathcal{S}, \mathcal{O}; \Phi, \Pi, \mathcal{T}, H)$. An external input or initial condition initializes the state, the transition protocol forms a trajectory, and the readout map observes the resulting state.
Transition protocol Π	The protocol that determines how transition rules are applied to system configurations. It may be a fixed schedule, recurrent iteration rule, conditional application rule, interaction protocol, or termination protocol. Π governs trajectory formation; CEU factorizes an individual transition.
Transition family $\mathcal{T}$	A transition rule or family of transition rules available to the system. A fixed-depth neural network may use a sequence of layer-wise transition rules; an adaptive or interactive system may select among transition rules under Π .
Computational configuration χ	The full configuration used by the transition protocol, which may include the system state, step or layer index, external signals, memory, context, environment state, or protocol variables. The internal state s is the part of the configuration transformed by the transition rule.
State-space dynamics	The state evolution induced by a transition map T , with the unconditioned form $s_{t+1} = T(s_t)$.
DAWN-CEU state-transition formulation	A transition-factorization formulation over general state-transition systems. It factorizes individual transition rules into Control, Execution, and Update maps; it does not define the global transition protocol.
CEU decomposition	The decomposition of an individual transition rule into a control rule C , an execution rule E , and an update rule U : $u_t = C(s_t)$, $z_t = E(s_t, u_t)$, and $s_{t+1} = U(s_t, z_t)$.
Operator-Coordinate CEU	A CEU specialization in which $C : \mathcal{S} \rightarrow \mathcal{Z}_{\text{op}}$ maps a representation state into an operator-coordinate space, and the coordinate parameterizes a sparse execution relation over a transition-operator family.
Learned DAWN-CEU instance	A parameterized instance with C_{θ_C} , E_{θ_E} , and optionally U_{θ_U} . Rules fixed by architecture correspond to empty or singleton parameter spaces.
Notation convention	Subscripts θ indicate learned parameterization; fixed architectural rules are written without θ .
State space $\mathcal{S}$	The abstract system-state space. In DAWN-SRW, this is instantiated as the model's residual representation space, called the residual state space.
Execution-specification space $\mathcal{U}_{\text{spec}}$	The space of possible execution specifications. The subscript avoids confusion with the update rule U .
Operator-coordinate space $\mathcal{Z}_{\text{op}}$	The coordinate space used by Operator-Coordinate CEU. A coordinate in this space does not directly produce the next state; it parameterizes a state-to-operator execution relation.
Execution-result space $\mathcal{Z}$	The space of execution results produced before the update rule is applied.
Current system state s_t	The general state read by the control rule and updated through the update rule. In DAWN-SRW, this is instantiated as the residual state x_t .
Residual state x_t	The current residual-stream vector at a computation step, layer, or token position; the DAWN-SRW instantiation of s_t .
Execution specification u_t	The output of the control rule. In DAWN-SRW, u_t is the set of participating RW operator identities and gates.
Execution result z_t	The output of the execution rule before the update rule is applied. In DAWN-SRW, $z_t = \Delta x_t$.
Induced execution function f_u	For a fixed execution specification u , the function $f_u : \mathcal{S} \rightarrow \mathcal{Z}$ defined by $f_u(s) = E(s, u)$.
Space traceability	Exposure of CEU-space objects, such as $u \in \mathcal{U}_{\text{spec}}$ or $z \in \mathcal{Z}$, in identifiable form.
Function traceability	Exposure of the induced execution function f_u as a composition of identifiable component functions.
Control rule C / C_{θ_C}	The rule that maps the current system state to an execution specification. In DAWN-SRW: Select.
Execution rule E / E_{θ_E}	The rule that uses the current system state and execution specification to produce an execution result. In DAWN-SRW: RWCompose.
Update rule U / U_{θ_U}	The rule that maps the current system state and execution result to the next system state. In DAWN-SRW, this is the fixed residual update $U(x_t, z_t) = x_t + z_t$.
Residual-state transition	DAWN-SRW transition between residual states, $x_t \rightarrow x_{t+1}$, with $x_{t+1} = x_t + z_t$ and $z_t = \Delta x_t$.

DAWN abstraction and DAWN-SRW implementation.

Term	Meaning
DAWN unit / neuron	An identifiable computational entity with a control-facing signature and an execution-facing operation.
Signature	The conceptual control-facing identity of a DAWN unit. In final DAWN-SRW, this identity is tied to the RW operator through an RW-derived operator key.
RW-derived operator key k_i	The Select-facing key of an RW operator, derived from its read/write directions as $k_i = K(r_i, w_i)$.
Legacy route embedding e_i	The independent route embedding used in the reported v4.1.5.5 implementation. It is an implementation detail of the reported experiments, not the conceptual operator identity in the final formulation.
Operation	The execution-facing computation performed when the unit participates.
RW neuron	The DAWN-SRW implementation of a DAWN unit as an explicit read/write transition operator, parameterized by a read vector r_i , a write vector w_i , and a Select-facing operator key k_i .
RW operation	The rank-1 read/write map $O_i^{RW}(x) = \langle x, r_i \rangle w_i$.
State-to-operator execution relation	The sparse relation induced by Select between residual states and RW transition operators.
Routing-induced execution domain	For operator W_i , the soft domain $\Omega_i^\epsilon = \{x \in \mathcal{S} \mid \gamma_i(x) > \epsilon\}$. These domains can overlap and are not assumed to form a disjoint partition.
Select	The DAWN-SRW instantiation of the control rule: a learned differentiable control rule whose output is an observable execution specification u_t identifying participating RW operator identities and gates.
RWCompose	The DAWN-SRW instantiation of the execution rule: it applies selected RW operators and composes traceable rank-1 read/write contributions into the execution result $z_t = \Delta x_t$.

Operator-page terms.

Term	Meaning
Global CEU operator page	A fixed-size candidate block over the global RW-operator atlas. Its live centroid summarizes the current RW-derived operator keys in that block for coarse retrieval; semantic compactness is an empirical property, not assumed by definition. A page is not itself an expert and does not perform computation.
Page centroid	A live representative coordinate for a page, computed from the RW-derived operator keys in that page. It is used for coarse page scoring in operator space.
Page-routed candidate set	The temporary union of operators in the selected top- K pages. DirectTau admission and RWCompose execution are applied inside this set.
Microgroup page selection	A hardware-oriented implementation option in which a small group of sequences shares the selected page set, while each token keeps its own operator query and token-specific operator weights inside the selected pages.

Layer terms.

Term	Meaning
Attention Layer	Constructs Q, K, and V through Select and RWCompose. QK attention determines how token states interact, and value aggregation carries the relational information flow.
RST Layer	The post-attention Residual State Transition layer. It applies Select and RWCompose over the RST pool to produce the residual state change; the residual update completes the residual-state transition.
Dense transition	Dense layers can be written as state transitions, but usually do not produce an observable execution specification u_t .
Trace condition	An execution specification is useful for analysis when it is identifiable, input-dependent, and directly tied to executed contributions.

Execution and evaluation terms.

Term	Meaning
Full soft-gated evaluation	The default DAWN-SRW evaluation: all RW-neuron pools are evaluated with differentiable gates. This is the implemented full-pool/chunked computation path.
Active RW neuron	A RW neuron whose Select activation exceeds 0.5. This is a broad candidate-participation diagnostic.
Strong RW neuron	A RW neuron whose Select activation exceeds 0.9. This is the high-confidence participation diagnostic; the main hard-pruned result uses the stricter threshold $\rho = 0.99$.
Hard-pruned evaluation	An evaluation-only forward pass in which RW neurons below a chosen activation threshold have their gates set to zero and RWCompose is renormalized over the retained gates.
Retained gate mass	The fraction of full soft-gated gate mass retained after thresholding: $\sum_i m_i g_i / (\sum_i g_i + \epsilon)$.
Estimated sparse-execution FLOPs	Theoretical FLOPs under a selected-RW execution accounting convention. These estimates are not measured wall-clock speedups of the current dense/chunked implementation.

A.3. CEU Traceability Formalization

This subsection makes explicit the traceability concepts used in the main text. It is not needed to define DAWN-SRW, but it clarifies how the DAWN-CEU formulation separates spaces, maps, transitions, induced execution functions, and traceability.

Spaces. Let $\mathcal{S}$, $\mathcal{U}_{\text{spec}}$, and $\mathcal{Z}$ denote the system-state space, execution-specification space, and execution-result space.

Maps. A CEU transition is defined by maps

$$C : \mathcal{S} \rightarrow \mathcal{U}_{\text{spec}}, \quad E : \mathcal{S} \times \mathcal{U}_{\text{spec}} \rightarrow \mathcal{Z}, \quad U : \mathcal{S} \times \mathcal{Z} \rightarrow \mathcal{S}.$$

Transition. Given $s \in \mathcal{S}$, the transition proceeds as

$$u = C(s), \quad z = E(s, u), \quad s' = U(s, z),$$

with factorized state-transition function

$$T(s) = U(s, E(s, C(s))).$$

Induced execution function. For a fixed execution specification $u \in \mathcal{U}_{\text{spec}}$, the execution map E induces an execution function

$$f_u : \mathcal{S} \rightarrow \mathcal{Z}, \quad f_u(s) = E(s, u).$$

Thus the transition can also be followed as

$$s \mapsto u = C(s) \mapsto f_u \mapsto z = f_u(s) \mapsto s' = U(s, z).$$

State-transition traceability. We distinguish two forms. *Space traceability* concerns whether objects in the CEU spaces, such as $u \in \mathcal{U}_{\text{spec}}$ and $z \in \mathcal{Z}$, are exposed in identifiable form. *Function traceability* concerns whether the execution function f_u induced by u is exposed as a composition of identifiable component functions:

$$f_u(s) = \sum_{i \in A(u)} \alpha_i(u) f_i(s).$$

The CEU decomposition by itself does not guarantee interpretability; it only identifies where specification, execution, and update structure may be exposed or hidden. DAWN-SRW instantiates this traceable case over the residual state space: Select exposes u_t as participating RW neurons and gates, RWCompose exposes f_{u_t} as a composition of read/write component functions, and the fixed residual update applies the resulting execution result.

A.4. Dense Attention and the Interpretability Boundary

The DAWN-CEU formulation is not restricted to DAWN-SRW. It can also describe standard dense transformations and dense attention. This subsection makes explicit how dense attention fits into the formulation and where its interpretability boundary lies.

Dense maps as transitions. A dense map is a state transition in the broad sense: it maps a current state to a new state. Let $x \in \mathbb{R}^{d_{\text{in}}}$ and let

$$M \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}.$$

A dense transition is

$$y = Mx.$$

Under the DAWN-CEU formulation, this corresponds to a transition with degenerate control over a singleton null-specification space:

$$\mathcal{U}_{\text{spec}} = \{\emptyset\}, \quad C_{\emptyset}(x) = \emptyset, \quad E_M(x, \emptyset) = Mx, \quad U_{\text{rep}}(x, z) = z.$$

Thus,

$$x_{t+1} = U_{\text{rep}}(x_t, E_M(x_t, \emptyset)) = Mx_t.$$

When the input and output spaces coincide, the same map can be algebraically written in residual form as

$$Mx = x + (M - I)x,$$

but this is only a residual reparameterization of a learned dense transformation. The dense map itself does not expose an execution specification analogous to u_t .

Dense maps at the linear-combination level. Although a dense map does not expose a nontrivial control rule, it can be mechanically decomposed at the linear-algebraic level. Writing the columns of M as

$$M = [m_1, \dots, m_{d_{\text{in}}}],$$

we have

$$Mx = \sum_{p=1}^{d_{\text{in}}} x_p m_p.$$

This exposes a coordinate-column contribution trace:

$$\text{contrib}_p(x) = x_p m_p.$$

Thus, at the linear-combination level, one can inspect how much each input coordinate contributes along the corresponding learned column direction. However, these coordinate-column contributions are not necessarily semantically identifiable computational units. Dense representations may distribute a single meaningful feature across many coordinates, and a single coordinate or column direction may participate in multiple functions. Therefore, dense maps are mechanically decomposable at the linear level, but their lower-level contributions remain semantically entangled.

Dense feed-forward layers. A dense feed-forward layer also admits a read/write-like algebraic form. For

$$W_2 \sigma(W_1 x),$$

write the rows of W_1 as $r_i^\top$ and the columns of W_2 as w_i . Then

$$W_2 \sigma(W_1 x) = \sum_i \sigma(\langle x, r_i \rangle) w_i.$$

This shows that dense feed-forward layers can be mechanically written as component functions. However, the component identities are not exposed by a separate execution specification, and the resulting components remain entangled in dense hidden coordinates.

Dense attention under a hierarchical CEU view. The CEU view can be applied hierarchically: a higher-level transition may contain subcomputations that themselves admit CEU descriptions. Standard dense attention can be written as a higher-level transition built on top of dense maps. Let $X = (x_1, \dots, x_n)$ denote the token-state sequence, with $x_i \in \mathbb{R}^{d_{\text{model}}}$. Dense projections construct attention interface states:

$$q_i = M_Q x_i, \quad k_j = M_K x_j, \quad v_j = M_V x_j.$$

The attention-level routing specification is

$$u_{ij}^{\text{attn}} = \text{softmax}_j \left(\frac{q_i^\top k_j}{\sqrt{d_h}} \right).$$

The execution rule aggregates routed value content:

$$z_i^{\text{attn}} = M_O \left(\sum_j u_{ij}^{\text{attn}} v_j \right),$$

and the standard Transformer residual update gives

$$x'_i = x_i + z_i^{\text{attn}}.$$

Substituting the dense projections, the full attention transition is

$$x'_i = x_i + M_O \left(\sum_j \text{softmax}_j \left(\frac{(M_Q x_i)^\top (M_K x_j)}{\sqrt{d_h}} \right) M_V x_j \right).$$

In DAWN-CEU terms, dense attention exposes an attention-level routing specification through query-key weights, but its Q/K/V interface construction is performed by implicit learned dense maps rather than by an exposed function-level execution trace.

Expanding the query-key interaction. At the attention-interface level, one can observe which query vector interacts with which key vector. At the dense-linear level, the score can be expanded further. For the input-column decompositions of M_Q and M_K , write

$$M_Q = [m_1^Q, \dots, m_{d_{\text{model}}}^Q], \quad M_K = [m_1^K, \dots, m_{d_{\text{model}}}^K].$$

Then

$$q_i = \sum_p x_{i,p} m_p^Q, \quad k_j = \sum_r x_{j,r} m_r^K.$$

The query-key score becomes

$$q_i^\top k_j = \left(\sum_p x_{i,p} m_p^Q \right)^\top \left(\sum_r x_{j,r} m_r^K \right),$$

or

$$q_i^\top k_j = \sum_{p,r} x_{i,p} x_{j,r} m_p^{Q\top} m_r^K.$$

This shows that a dense attention score is an aggregate interaction between two already-superposed dense interfaces. The contribution of coordinate p of token i and coordinate r of token j can be mechanically computed through

$$x_{i,p} x_{j,r} m_p^{Q\top} m_r^K.$$

However, this decomposition does not imply that the coordinate pair (p, r) corresponds to a stable semantic relation. It is a linear contribution decomposition, not an architecturally exposed function-level execution trace.

Value aggregation and output mixing. The value pathway has the same structure. If

$$M_V = [m_1^V, \dots, m_{d_{\text{model}}}^V],$$

then

$$v_j = \sum_r x_{j,r} m_r^V.$$

The contextual content received by token i is

$$c_i = \sum_j u_{ij}^{\text{attn}} v_j = \sum_{j,r} u_{ij}^{\text{attn}} x_{j,r} m_r^V.$$

Thus, the attention execution can be mechanically decomposed into routed coordinate-column value contributions. The output projection M_O then mixes these contributions into the residual update:

$$z_i^{\text{attn}} = M_O c_i.$$

Again, this operation is decomposable algebraically, but the resulting residual update superposes the routed value contributions in the output representation.

Interpretability boundary. This analysis localizes the interpretability boundary of dense attention. At the attention level, the interaction is observable: one can inspect query-key scores, attention weights, routed values, aggregated content, and residual updates. At the dense-linear level, Q/K/V formation can be decomposed into coordinate-column contributions. However, these contributions are immediately summed into dense interface vectors and need not correspond to semantically identifiable computational units. Therefore, standard dense attention is interpretable at the attention-routing level and mechanically decomposable at the dense-linear level, but remains semantically entangled at the level of interface construction.

In contrast, DAWN-SRW exposes both space and function traceability. Select produces an observable execution specification over RW transition operators, and RWCompose produces an execution result through an induced execution function that decomposes into identifiable read/write component functions. Thus, DAWN-SRW moves part of the computation that is implicit in dense Q/K/V formation into an exposed Select/RWCompose pathway.

B. Current Operator-Key and Page Implementation Notes

This appendix summarizes the implementation path corresponding to the current method section. The purpose is to separate the completed v4.1.6.6 operator-key path, the v4.1.6.7 page-routed extension, and the older v4.1.5.5 legacy route-embedding pilot.

B.1. v4.1.6.6 Operator-Key DirectTau Path

The v4.1.6.6 path removes the independent legacy route embedding from the conceptual Select identity. Each operator key is derived from the operator’s own read/write directions, and each token produces an RW-matched operator query. Select compares the query with RW-derived operator keys by cosine similarity, maps the resulting margin through DirectTau admission and boundary-relative drive, and passes the resulting execution weights to RWCompose.

In the completed 40M run, this path is evaluated in full-pool form: the model still computes scores over the complete operator pool, while execution-pruning evaluations zero weak execution weights according to an evaluation threshold. Thus the v4.1.6.6 result validates trainability and pruning stability of the final operator-key formulation, not implemented sparse wall-clock efficiency.

B.2. v4.1.6.7 Global CEU Operator Pages

The v4.1.6.7 path keeps the v4.1.6.6 operator-key, DirectTau, and RWCompose equations, but introduces a page-routed candidate set before operator-level selection. Operators are grouped into fixed-size pages. These pages are implementation-level candidate blocks, not assumed learned clusters. A live page centroid is computed from the RW-derived operator keys in each block. A page routing pass selects the top-scoring pages by centroid similarity in operator space and constructs a dense candidate matrix from the operators in those pages.

The page mechanism should not be interpreted as Mixture-of-Experts routing. A page is a fixed-size candidate block over the global operator atlas, not a computational expert. It only restricts the candidate set. Inside that set, each token still has its own operator query, tau boundary, admission, drive, read scalar, and write-vector contribution. In the current implementation, page selection can be shared by a microgroup for hardware efficiency, but the operator weights inside selected pages remain token-specific.

Future implementation work includes learned page anchors, page compactness or separation objectives, periodic operator repacking, load-balance analysis, hardware-specific read/write kernels, and direct wall-clock benchmarking.

C. Legacy v4.1.5.5 Implementation Details

This appendix gives the implementation-level parameterization used by the legacy DAWN-SRW v4.1.5.5 preliminary pilot. These details instantiate the reported experimental gate $g_i^p(x)$, corresponding to the abstract soft gate $\gamma_i^p(x)$ in the main text. They are not separate architectural contributions. The main architectural objects remain Select, RWCompose, and the observable execution specification u_t .

C.1. Layer Flow and Shared RW-Neuron Pools

Each DAWN-SRW block consists of an Attention Layer followed by an RST Layer. The Attention Layer constructs Q, K, and V through Select and RWCompose over attention RW-neuron pools, then applies standard causal self-attention and a dense output projection. The RST Layer applies Select and RWCompose over the RST pool to produce the post-attention residual state change.

The RW-neuron pools are shared across layers. In the reported v4.1.5.5 implementation, each pool contains legacy route embeddings, read vectors, write vectors, and a learned pool scale:

$$\{e_i^p, r_i^p, w_i^p\}_{i=1}^{N_p}, \quad \alpha_p. \quad (79)$$

The legacy route embedding e_i^p implements the Select-facing signature used by the reported experiments; the read/write vectors (r_i^p, w_i^p) implement the execution-facing RW operation. In the final v4.1.6.6 conceptual formulation, this independent route embedding is replaced by an RW-derived operator key $k_i^p = K_\theta^p(r_i^p, w_i^p)$, so the control-facing identity is tied to the execution-facing read/write map. Separate Select passes are used for Q and K, while Q and K share the attention-QK pool.

C.2. Route-Space Scoring

For each pool p , DAWN-SRW forms a route query

$$h^p(x) = W_{\text{route}}^p x + b_{\text{route}}^p, \quad (80)$$

and scores each RW neuron by matching this query against the legacy route embedding:

$$s_i^p(x) = h^p(x)^\top e_i^p. \quad (81)$$

This route-space scoring is the implementation-level v4.1.5.5 mechanism by which Select compares the current residual state with reusable RW-neuron signatures. It should be read separately from the final operator-coordinate formulation, where routing compares an operator-coordinate query against RW-derived operator keys.

C.3. Pool-Relative Threshold and Bounded Scan Offset

Select uses a pool-relative threshold so that participation is determined relative to the score distribution for the current token and pool. Let

$$\mu_s^p(x) = \frac{1}{N_p} \sum_{j=1}^{N_p} s_j^p(x), \quad (82)$$

$$\sigma_s^p(x) = \sqrt{\frac{1}{N_p} \sum_{j=1}^{N_p} (s_j^p(x) - \mu_s^p(x))^2 + \epsilon_\sigma}. \quad (83)$$

The bounded scan offset is

$$\delta_{\text{scan}}^p(x) = c_{\text{scan}} \tanh(\omega_{\text{scan}}^p(x)), \quad (84)$$

and the pool-relative threshold is

$$\tau^p(x) = \mu_s^p(x) + \lambda_\tau^p(x) \sigma_s^p(x) - \frac{\delta_{\text{scan}}^p(x)}{\max(\sigma_s^p(x), \sigma_{\min})}. \quad (85)$$

The tau offset controls the relative participation boundary in score-standard-deviation units. The bounded scan term allows a small local shift of this boundary while preventing unbounded threshold movement, especially when the score standard deviation is small.

C.4. Two-Axis Gate: Activation and Intensity

After thresholding, DAWN-SRW separates participation into two factors: activation and intensity. Define the raw margin and activation margin as

$$z_i^p(x) = s_i^p(x) - \tau^p(x), \quad (86)$$

$$m_i^p(x) = z_i^p(x) - a_0. \quad (87)$$

The activation term is

$$a_i^p(x) = \sigma(\beta m_i^p(x)), \quad (88)$$

and the intensity term is

$$\eta_i^p(x) = \epsilon + \min(\max(m_i^p(x) - c_0, 0), \eta_{\max}). \quad (89)$$

The final gate used by RWCompose is

$$g_i^p(x) = a_i^p(x) \eta_i^p(x). \quad (90)$$

The activation term acts as a near-binary participation indicator. In the v4.1.5.5 experiments, the sigmoid sharpness β is large, so activation rapidly switches around the participation boundary. The intensity term controls how strongly a participating RW neuron contributes once it is beyond the boundary. Thus, activation answers whether an RW neuron participates, while intensity controls the strength of the executed read/write contribution.

This two-axis gate is useful for analysis: active and strong sets are defined from activation, while the executed contribution depends on the full gate $g_i^p(x) = a_i^p(x) \eta_i^p(x)$ together with the read scalar and write direction.

C.5. Active and Strong RW Neurons

We use activation-based diagnostic sets:

$$\mathcal{A}^p(x) = \{i : a_i^p(x) > 0.5\}, \quad (91)$$

$$\mathcal{S}^p(x) = \{i : a_i^p(x) > 0.9\}. \quad (92)$$

The active set is a broad candidate-participation set. The strong set contains high-confidence participating RW neurons; the main hard-pruned result uses an even stricter threshold, $\rho = 0.99$. These sets are diagnostics of Select; they are distinct from ranking RW neurons by executed contribution, which additionally depends on read values, write directions, gates, and normalization.

C.6. RWCompose and Gate Normalization

Given gates $g_i^p(x)$, RWCompose forms

$$y_p(x) = \alpha_p \frac{\sum_{i=1}^{N_p} g_i^p(x) \langle x, r_i^p \rangle w_i^p}{\max\left(\sum_{i=1}^{N_p} g_i^p(x), 1\right)}. \quad (93)$$

The denominator uses the same activation-weighted intensity that appears in the numerator. It prevents unstable amplification when very few neurons participate while still allowing broad combinations of RW-neuron contributions. In the RST Layer, this output is the residual state change. In attention pools, Select and RWCompose construct Q, K, or V projections before causal self-attention.

Quantity	DAWN-SRW	Dense baseline
Parameters	40.221M	40.344M
Layers	12	12
Hidden dimension d	384	432
FFN dimension	–	1,728
Routing dimension d_r	128	–
Attention heads	6	6
RW-neuron pools	1,480 / 7,330 / 20,560	–
Vocabulary	30,522	30,522
Sequence length	512	512
Training tokens	5B	5B
Paper-eval loss	3.62589	3.60568
Paper-eval accuracy	35.153%	35.397%

Table 5. 40M language-modeling setup. The dense baseline is parameter-matched and uses a larger residual width with a standard $4\times$ FFN.

Pool	Embedding	Read	Write
Attention-QK	0.8028	3.4099	2.7556
Attention-V	1.0943	1.8823	1.7216
RST	1.1631	1.7592	1.7550

Table 6. Mean RW-neuron vector norms for the trained 40M DAWN-SRW model.

C.7. Dense/Chunked Full-Pool Evaluation

The present implementation evaluates scores and gate-weighted read/write contributions over the full RW-neuron pools using chunked dense computation. This implementation choice prioritizes differentiable training and architectural clarity. Consequently, active and strong RW-neuron counts should not be interpreted as measured wall-clock speedups of the current implementation. They characterize the sparsity of the learned execution specification.

The hard-pruned evaluations in Appendix E are actual forward passes with gates zeroed below an activation threshold, but the FLOPs reported there remain theoretical sparse-execution estimates unless otherwise stated.

D. Experimental Setup and Model Configurations

The primary 40M validation in this draft is the v4.1.6.6 operator-key run. It uses the same 5B-token C4 budget and 76,293 training steps as the dense Transformer baseline. The completed run has 40.220M parameters and reaches end-of-epoch validation loss 3.6553 and accuracy 34.73%. The dense Transformer baseline has 40.344M parameters and reaches validation loss 3.5986 and accuracy 35.44%. The older v4.1.5.5 configuration below is retained because it is the source of the legacy hard-pruning and trace-analysis pilot.

Table 5 gives the full 40M DAWN-SRW configuration and the dense Transformer baseline used for language-modeling comparison. The baseline is parameter-matched and uses a larger residual width with a standard $4\times$ FFN. Table 6 reports mean RW-neuron vector norms after training; these diagnostics are not treated as main results, but confirm that the migrated checkpoint loads correctly and that the renamed RW-neuron pools remain populated.

The effective ranks of the attention-QK, attention-V, and RST legacy route embeddings are 120.0, 122.9, and 122.1 respectively out of a 128-dimensional route space.

E. Hard-Pruned Evaluation and FLOPs Accounting

E.1. Hard-Pruned Forward Definition

The hard-pruned evaluation in this appendix belongs to the legacy v4.1.5.5 preliminary pilot. It tests whether the learned soft-gated computation is concentrated in high-confidence selected RW neurons. For an activation threshold ρ , define the retained mask

$$M_i^{p,\rho}(x) = \mathbf{1}[a_i^p(x) > \rho]. \quad (94)$$

DAWN-SRW: Computational Model of Learned State Transitions

Compute regime	Core FLOPs/token	Total incl. LM head	Ratio vs TF	Ratio vs DAWN full	Notes
Dense Transformer	59.07M	85.44M	1.000	—	dense baseline
DAWN-SRW full pool	676.38M	699.82M	11.451	1.000	current code path
Exact routing + hard .99 RW	180.53M	203.97M	3.056	0.267	full routing, pruned RW
Selected-RW only .99	72.77M	96.22M	1.232	0.128	idealized target

Table 7. Theoretical FLOPs accounting for the 40M v4.1.5.5 experiment. Ratios and FLOPs are accounting estimates; they are reported separately from measured validation loss and are not wall-clock speedups.

The pruned gate is

$$\tilde{g}_i^{p,\rho}(x) = M_i^{p,\rho}(x)g_i^p(x), \quad (95)$$

and the hard-pruned RWCompose output is

$$\tilde{y}_p^\rho(x) = \alpha_p \frac{\sum_i \tilde{g}_i^{p,\rho}(x) \langle x, r_i^p \rangle w_i^p}{\max(\sum_i \tilde{g}_i^{p,\rho}(x), 1)}. \quad (96)$$

Thus, hard-pruned evaluation is an actual forward pass with weak gates removed, not merely a diagnostic count. The diagnostic strong-neuron threshold is $\rho = 0.9$. The main text reports a stricter representative setting, $\rho = 0.99$, over all RW-neuron pools.

E.2. Evaluation Variants

The main text reports the full soft-gated model and a strict representative hard-pruned evaluation at $\rho = 0.99$. The full appendix sweep evaluates $\rho \in \{0.50, 0.70, 0.80, 0.90, 0.95, 0.99\}$ over all pools. We also reserve optional variants for RST-only, attention-only, QK-only, and V-only pruning to distinguish which component is most sensitive to hard execution.

For each variant, we report validation cross-entropy, accuracy, retained RW-neuron counts, retained gate mass, and theoretical sparse-execution FLOPs. The validation loss is always the language-modeling cross-entropy; auxiliary losses are reported only as training diagnostics in Appendix F.

E.3. FLOPs Accounting Convention

We separate three compute accounting regimes. First, *implemented full-pool DAWN-SRW* evaluates routing, scores, gates, and read/write composition over the full RW-neuron pools using dense/chunked computation. Second, *exact-routing sparse execution* assumes the model still computes full-pool routing scores, but executes read/write operations only for the retained RW neurons. Third, *ideal selected-RW execution* counts only selected read/write execution and should be read as an implementation target rather than a property of the current code.

All FLOPs use the convention that a multiply-add counts as two FLOPs. For a pool with N_p RW neurons, route dimension d_r , residual dimension d , and retained count k_p^ρ , full score computation costs approximately $2N_p d_r$ FLOPs per token per route. We count one retained RW operation as approximately $4d$ FLOPs: $2d$ for the read dot product and $2d$ for writing/accumulating the scaled write vector. Thus, full-pool and pruned RW costs are

$$F_{p,\text{RW}}^{\text{full}} \approx 4N_p d, \quad (97)$$

$$F_{p,\text{RW}}^\rho \approx 4k_p^\rho d. \quad (98)$$

For attention Q/K construction, the QK pool is used through separate Q and K Select/RWCompose passes; V and RST use their corresponding pools. Attention interaction and any dense output projections are reported separately from RW-neuron execution.

F. Sweep Results

F.1. Hard-Pruning Threshold Sweep

The threshold sweep evaluates whether the soft-gated computation remains concentrated in high-confidence selected RW neurons as the activation threshold increases. The main text reports the strict threshold $\rho = 0.99$; the full sweep is reserved for this appendix.

Variant	ρ	Val loss	Δ loss	Acc.	QK kept	V kept	RST kept	Gate mass	RW-only
full soft	–	3.62589	0.00000	35.153	1,480	7,330	20,560	1.000	1.000
hard all	0.50	3.62589	-0.00001	35.153	110.32	706.51	3,049.23	1.000	0.132
hard all	0.70	3.62590	0.00000	35.153	109.99	704.68	3,043.63	1.000	0.131
hard all	0.80	3.62590	0.00001	35.153	110.23	706.20	3,052.16	1.000	0.132
hard all	0.90	3.62589	-0.00001	35.153	109.91	704.45	3,046.79	1.000	0.131
hard all	0.95	3.62589	-0.00001	35.153	109.49	702.77	3,038.04	1.000	0.131
hard all	0.99	3.62590	0.00000	35.152	108.99	699.31	3,030.97	1.000	0.131

Table 8. Hard-pruned threshold sweep. Gate mass is reported from the evaluation table rounded to three decimals; the unchanged loss indicates that the soft-gated computation is concentrated in high-confidence RW neurons. RW-only is the estimated selected-read/write execution ratio, not measured wall-clock speedup.

Run	Weight	Asym.	Full loss	Strong loss	Δ loss	QK strong	V strong	RST strong	RW-only
tag128	0.001	1.5	3.63294	3.63294	-0.00001	122.00	386.13	3,381.57	0.133
ex10	0.001	10.0	3.66154	3.66153	-0.00001	55.35	82.78	527.53	0.023
ex20	0.001	20.0	3.66953	3.66955	0.00001	57.00	86.32	564.89	0.024

Table 9. Exploration-asymmetry sweep. Strong-loss rows use hard-pruned strong-only evaluation for the corresponding checkpoint. Higher asymmetry produces much sparser selected RW execution but increases validation loss; these runs are diagnostic and are not used as the primary language-modeling comparison.

F.2. Exploration-Loss Sweep

The main text reports validation cross-entropy and accuracy. Exploration-loss and asymmetry sweeps are used only to characterize the sparsity–performance trade-off and are therefore reported in the appendix. For each sweep checkpoint, we evaluate the full soft-gated model and a specified hard-pruned variant. The reported validation loss is the language-modeling cross-entropy; auxiliary exploration losses are training diagnostics and are not used as the primary comparison metric.

G. RW-Neuron Trajectory Analysis Details

For context-dependence experiments, we compare RST control and execution traces using three RW-neuron sets at each layer.

Active set. The active set contains RW neurons whose activation exceeds a threshold:

$$A_{\ell,t} = \{i : a_i > 0.5\}. \quad (99)$$

This measures the broad candidate set participating in Select’s control trace.

Top- k projected contributors. For each RW neuron, we compute its contribution vector

$$c_i = \alpha_p \frac{g_i O_i^{RW}(x)}{\max(\sum_j g_j, 1)}. \quad (100)$$

We then rank RW neurons by their signed projection onto the composed transition output $y_p(x)$:

$$\pi_i = \left\langle c_i, \frac{y_p(x)}{\|y_p(x)\| + \epsilon} \right\rangle. \quad (101)$$

The top- k contributor set contains the RW neurons with the largest projected contribution.

Effective-mass contributors. We also define an effective contributor set as the smallest set of RW neurons whose cumulative absolute projected contribution reaches a fixed fraction of the total mass. In our experiments, this threshold is 80%.

For two contexts containing the same surface token, we compare these sets layer by layer using Jaccard overlap:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}. \quad (102)$$

This lets us distinguish broad candidate overlap from overlap among the RW neurons that actually dominate the RST transition.

G.1. Bank Case Study

We examine *bank* more closely using the contexts *I deposited money in the bank* and *He sat by the river bank*. Figure 2 shows both the layer-wise overlap and the output-norm comparison for this token. The two contexts start with high overlap in the early layers, but the top- k and effective-mass contributor sets separate strongly in the middle layers. For example, top- k Jaccard overlap falls near zero around layers 4–5, while the active-set overlap remains substantially higher.

The norm comparison shows that the RST transition output grows in later layers and becomes especially large in the final layer. This supports the interpretation that middle layers separate context-dependent RW-neuron sets, while later layers amplify the context-specific residual state transition.

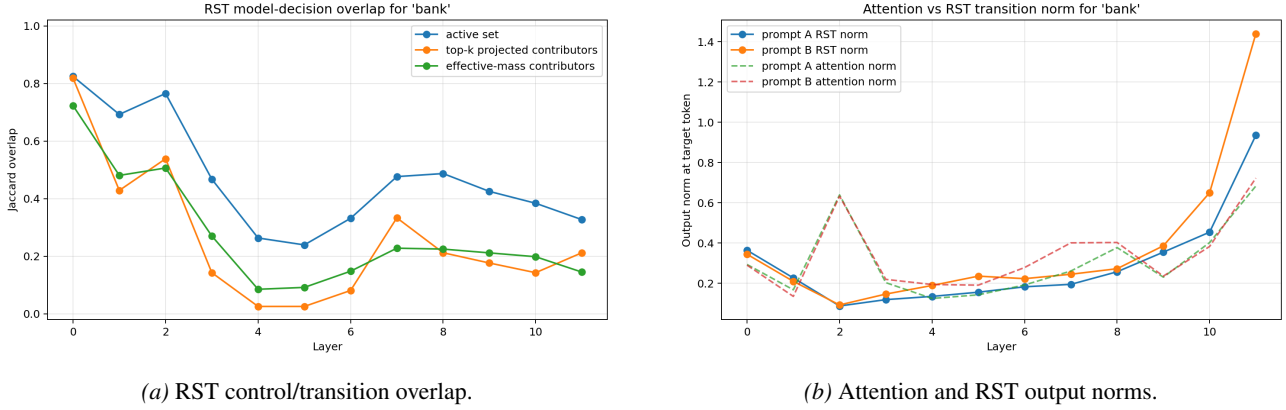


Figure 2. Case study for the initially underdetermined token state *bank*. The financial and river-bank contexts share many broad RST candidates in early layers, but their high-contribution and effective-mass contributor sets separate in the middle layers. In later layers, the RST transition norm grows, indicating a stronger context-specific residual state transition.

G.2. RW-Neuron Execution Trace

Because each RST RW neuron has an explicit read/write transition, we can trace individual contributions to the composed transition. Table 10 shows the top projected contributors for *bank* in the final layer under the financial context. The table separates routing evidence (score, activation, intensity, gate) from RW-neuron evidence (read value, contribution norm, projection onto the final RST transition, and cosine alignment).

Rank	Neuron	Score	Act.	Int.	Gate	Read	Contrib.	Projection / Align.
1	9537	12.917	1.000	8.775	8.775	10.393	0.1466	0.0578 / 0.394
2	4504	10.740	1.000	6.598	6.598	-7.487	0.0968	0.0385 / 0.397
3	216	11.512	1.000	7.370	7.370	7.253	0.1070	0.0310 / 0.290
4	4729	10.306	1.000	6.164	6.164	-8.440	0.0863	0.0308 / 0.357
5	15651	10.119	1.000	5.976	5.976	-8.826	0.0917	0.0298 / 0.325
6	16201	9.553	1.000	5.411	5.411	7.792	0.0850	0.0252 / 0.296
7	7272	16.177	1.000	12.035	12.035	-3.367	0.0768	0.0241 / 0.314
8	15419	8.702	1.000	4.560	4.560	-9.003	0.0757	0.0166 / 0.219

Table 10. RW-neuron execution trace for the token *bank* in the final RST Layer. Top contributors are ranked by signed projection onto the composed RST transition output. The effective role of an RW neuron is not determined by score alone; it depends on the full read/write contribution. Alignment is the cosine similarity between the individual contribution vector and the composed RST transition output.

The trace illustrates why Select control cannot be reduced to routing scores alone. An RW neuron with a high score may not dominate the transition if its read value or write direction contributes weakly to the composed output. Conversely, an RW neuron with a moderate score can become a high projected contributor when its read/write contribution aligns with the final transition direction. Thus, DAWN-SRW provides an observable selection output and traceable RW-neuron execution contributions that follow selection.

H. Formal Low-Rank Property

The read/write composition has a simple algebraic interpretation. For a fixed input state, a selected or thresholded pool output is equivalent to applying an input-conditioned low-rank matrix to the current state. The resulting map is not globally linear, since the gates and selected set depend on the input state. In implementations with soft gates, the exact dense-gated pool output is recovered by including all nonzero gates; the thresholded form below characterizes the effective or sparse executed transition used for analysis and sparse-execution accounting.

Proposition 1 (Pointwise input-conditioned low-rank transition). For a fixed input state x and threshold $\gamma \geq 0$, let

$$S_p^\gamma(x) = \{i : g_i(x) > \gamma\} \quad (103)$$

be the thresholded RW-neuron set in pool p . Define the thresholded pool output

$$y_p^\gamma(x) = \alpha_p \frac{\sum_{i \in S_p^\gamma(x)} g_i(x) O_i^{RW}(x)}{\max\left(\sum_{i \in S_p^\gamma(x)} g_i(x), 1\right)}. \quad (104)$$

Then

$$y_p^\gamma(x) = A_p^\gamma(x)x, \quad (105)$$

where

$$A_p^\gamma(x) = \frac{\alpha_p}{\max\left(\sum_{i \in S_p^\gamma(x)} g_i(x), 1\right)} \sum_{i \in S_p^\gamma(x)} g_i(x) w_i r_i^\top. \quad (106)$$

Therefore, for this thresholded executed composition,

$$\text{rank}(A_p^\gamma(x)) \leq |S_p^\gamma(x)|. \quad (107)$$

Proof. Using $O_i^{RW}(x) = \langle x, r_i \rangle w_i = w_i r_i^\top x$, the thresholded pool output is

$$\begin{aligned} y_p^\gamma(x) &= \frac{\alpha_p}{\max\left(\sum_{i \in S_p^\gamma(x)} g_i(x), 1\right)} \sum_{i \in S_p^\gamma(x)} g_i(x) w_i r_i^\top x \\ &= A_p^\gamma(x)x. \end{aligned} \quad (108)$$

Since $A_p^\gamma(x)$ is a scalar multiple of a sum of $|S_p^\gamma(x)|$ rank-1 matrices, its rank is at most $|S_p^\gamma(x)|$. The exact soft-gated output in Equation 38 is the special case obtained by taking $S_p^\gamma(x)$ to include all nonzero gates, in which case the rank bound becomes the number of included RW neurons, at most N_p .

Corollary 1 (Linear transitions as a special case). Any linear transition Ax with $\text{rank}(A) = r$ can be represented by an unnormalized sum of at most r RW neurons. With a fixed nonzero pool normalization, the resulting scalar can be absorbed into the write vectors or the pool scale.

Proof. By rank factorization, any rank- r matrix can be written as

$$A = \sum_{k=1}^r w_k r_k^\top. \quad (109)$$

Setting each RW neuron to $O_k(x) = \langle x, r_k \rangle w_k$ and selecting these r RW neurons gives

$$\sum_{k=1}^r O_k(x) = Ax. \quad (110)$$

Thus, a single RW neuron is rank-1, but compositions of RW neurons can represent higher-rank linear transitions; DAWN-SRW makes this composition state-dependent through Select and gates.

I. 400M Pilot with an Earlier SRW Variant

This appendix reports a 400M pilot using spatial-r1-v3.9.4, an earlier SRW-family implementation. The purpose of this pilot is narrow: to test whether RW-neuron composition can train at the 400M scale and produce transferable representations. We do not use this experiment as evidence for the v4.1.6.6 RW-derived operator-key formulation, the v4.1.6.7 page-routed mechanism, or implemented compute efficiency.

I.1. Relation to Later DAWN-SRW Implementations

Table 11 summarizes the implementation differences that matter for interpreting the 400M pilot. Both models use route-space scoring and read/write neuron composition, but v3.9.4 predates the final DAWN-SRW terminology and gate parameterization used in the main 40M experiments. In particular, v3.9.4 uses a ReLU gate with gate-sum normalization and a “knowledge” pool, whereas v4.1.5.5 uses the RST terminology and a two-axis activation–intensity gate with bounded scan offset.

Aspect	spatial-r1-v3.9.4 pilot	DAWN-SRW v4.1.5.5
Layer naming	Attention + knowledge layer; knowledge pool	Attention Layer + RST Layer; RST pool
Gate form	ReLU/linear gate, clipped and gate-sum normalized	Activation–intensity gate, multiplied into final gate
Threshold	Relative tau from score mean and std	Relative tau plus bounded scan offset
Read/write vectors	Forward-normalized read/write directions	Raw read/write vectors; norms are diagnostics
Sparsity evidence	Utilization diagnostics only; not final sparsity evidence	Full-soft and strict hard-pruned evaluation for final DAWN-SRW
Role in paper	400M scaling and transfer pilot	Main architecture and sparsity/trace analysis

Table 11. Key differences between the 400M spatial-r1-v3.9.4 pilot and the later DAWN-SRW implementations used for 40M operator-key validation, legacy hard-pruning, and page-routed development.

I.2. Pilot Result Summary

Table 12 summarizes the limited conclusions supported by the 400M pilot before the detailed pretraining, downstream, utilization, and compute tables. The pilot supports scaling and transfer of the SRW family, but not implemented sparse inference efficiency.

Question	Result	Interpretation
400M pretraining	DAWN v3.9.4 re-eval loss 2.9330 vs. dense Transformer 2.9588	SRW-family RW composition trains competitively at 400M scale
Downstream transfer	DAWN pretraining improves over random initialization on all five tasks	C4 pretraining learns transferable representations
Dense comparison	DAWN is higher on SST-2, RTE, and MNLI; lower on WiC and BoolQ	Broadly comparable, not uniform task dominance
Sparsity/efficiency	Large active fractions in V and knowledge pools; dense/chunked implementation	Not evidence for final sparse-efficiency claim

Table 12. Summary of what the 400M v3.9.4 pilot does and does not establish. Detailed task metrics and compute accounting are reported in Tables 13–17.

Fine-tuning protocol. All downstream evaluations use a single fine-tuning run per model and task. The shared protocol is AdamW with learning rate 2×10^{-5} , warmup ratio 0.06, weight decay 0.01, batch size 64, maximum sequence length 512, and one seed. Evaluation intervals are 100 steps for RTE, SST-2, and WiC; 150 steps for BoolQ; and 300 steps for MNLI.

I.3. Pretraining and Checkpoint Metrics

Table 13 summarizes the pretraining setup and checkpoint re-evaluation metrics. DAWN v3.9.4 reaches a validation loss of 2.9330, compared with 2.9588 for the dense Transformer baseline. This supports the limited claim that SRW-style RW-neuron composition can train competitively at the 400M scale.

DAWN-SRW: Computational Model of Learned State Transitions

Model	Params	Tokens	Seq. len	Ckpt step	Best val loss	Re-eval val loss / ppl
DAWN v3.9.4	399.5M	40B	512	76,293	2.9387	2.9330 / 18.78
Transformer	393.9M	40B	512	75,800	2.9701	2.9588 / 19.27

Table 13. 400M pretraining setup and checkpoint metrics. DAWN uses spatial-r1-v3.9.4, an earlier SRW-family implementation.

I.4. Downstream Transfer

Table 14 reports best downstream accuracy across five classification tasks. DAWN v3.9.4 pretraining improves substantially over random initialization on every task, with gains ranging from +11.28 to +32.63 percentage points. Relative to the dense Transformer baseline, the pretrained DAWN model is higher on SST-2, RTE, and MNLI, lower on WiC and BoolQ, and broadly comparable overall. We therefore interpret these results as evidence for transfer, not as a claim of uniform task dominance.

Task	DAWN pre.	DAWN rand.	DAWN Δ	Dense pre.	Dense rand.	Dense Δ	DAWN-Dense
SST-2	94.04	76.49	+17.55	93.69	75.57	+18.12	+0.35
RTE	71.48	51.26	+20.22	68.95	53.07	+15.88	+2.53
WiC	67.24	55.96	+11.28	69.75	56.27	+13.48	-2.51
BoolQ	77.52	65.63	+11.89	78.13	64.86	+13.27	-0.61
MNLI	82.76	50.13	+32.63	82.63	51.13	+31.50	+0.13

Table 14. Best downstream accuracy for the 400M pilot. Deltas are percentage-point differences. DAWN-Dense compares pretrained DAWN v3.9.4 against the pretrained dense Transformer baseline.

I.5. Utilization and Compute Diagnostics

The following diagnostics are included for transparency. They are not used as the main sparsity evidence for DAWN-SRW because v3.9.4 uses an earlier ReLU/gate-sum-normalized routing scheme and activates large portions of the RW-neuron pools, especially the V and knowledge pools.

Pool	Active frac.	Active ops/token
QK	0.0649	415.52
V	0.4024	4,185.48
Knowledge	0.2334	23,337.13

Table 15. Utilization diagnostics for the earlier v3.9.4 400M pilot.

Metric	Value
Gate-sum mean	2,852.3666
Gate-max mean	1.4163
Gate concentration mean	6.6673
Score-std mean	0.3087

Table 16. Additional routing diagnostics for spatial-r1-v3.9.4.

Table 17 separates implemented compute from theoretical active-RW-neuron estimates. The current spatial-r1-v3.9.4 implementation should be read as a dense-routing implementation: it computes routing scores over the full RW-neuron pools and also applies read/write operations over those pools before gate normalization. We therefore do not use this appendix as evidence for implemented compute efficiency. The active-read/write rows are theoretical accounting regimes: they ask how the cost would change if read/write execution, and then routing itself, were restricted to the measured active RW-neuron set. These estimates illustrate the algorithmic distinction between learned RW-neuron capacity and executed transition computation; they are not measured wall-clock results.

Compute regime	Core FLOPs/token	Total incl. LM head
Dense Transformer	7.550×10^8	8.331×10^8
v3.9.4 implemented dense routing	1.348×10^{10}	1.356×10^{10}
v3.9.4 dense routing + active RW	5.037×10^9	5.116×10^9
v3.9.4 route-gated active RW	3.289×10^9	3.367×10^9

Table 17. Approximate compute accounting for the 400M pilot. The implemented v3.9.4 row reflects the current dense-routing implementation. The active-RW rows are theoretical accounting estimates based on measured active counts; the sparse-routing row assumes candidate-restricted routing over the active set and is not a measured wall-clock result.

I.6. Interpretation

The 400M pilot supports two limited conclusions. First, an SRW-family model with RW-neuron composition can be trained at 400M scale and can reach competitive language-modeling validation loss. Second, C4 pretraining produces transferable representations, as shown by large gains over random initialization and downstream performance broadly comparable to a dense Transformer baseline. At the same time, the utilization and compute diagnostics show that spatial-r1-v3.9.4 is not yet an efficient sparse inference implementation. The final sparsity mechanism and compute-efficiency direction should therefore be evaluated using the later v4.1.6.6 operator-key path and the v4.1.6.7 page-routed implementation.